AI in Space: A First Autonomous Rendezvous Simulation

Fabio Facin

May 2026

Abstract

This note records the first working version of an AI-in-space project. The first problem is small on purpose: a chaser spacecraft in low Earth orbit tries to rendezvous with a target by choosing from five simple thrust actions. The goal is not to claim a finished autonomy system. The goal is to build a loop we can inspect: propagate the orbit, choose an action, measure the result, and compare that behavior against later learning agents. The current baseline uses a short-horizon scoring rule, which I call the Rendezvous Intent Functional. I also add an early tabular Q-learning scaffold and a randomized-scenario test, which immediately shows the difference between solving one setup and learning behavior that generalizes.

1 Motivation

The goal of this project is to build an extensible simulation environment where spacecraft autonomy can be tested under increasingly realistic physical constraints. I am not starting with a neural network because that would hide too much too early. The first version should be something I can understand, break, and improve without guessing what the model learned. For that reason, the first controller is a transparent lookahead rule. It is not elegant, and it is not final, but it gives the project a baseline that future agents have to beat.

This direction is useful because autonomous spacecraft will need to make decisions when direct human control is delayed, unavailable, or too slow. Examples include satellite inspection, debris avoidance, lunar logistics, asteroid missions, deep-space navigation, and high-power electric propulsion missions. The long-term version of this project should let an agent reason about orbit, fuel, power, heat, sensor uncertainty, communication delay, radiation, and mission risk. For now, the right move is to get the small rendezvous problem working cleanly.

2 First Scenario

The first scenario is planar autonomous rendezvous. There are two spacecraft:

- a target spacecraft in a circular orbit at altitude 500 km,
- a chaser spacecraft in a nearby circular orbit at altitude 485 km, initially behind the target by 0.045 rad.

This fixed setup is the starter case. After the first successful run, I added randomized scenarios where the target altitude, chaser altitude offset, and starting phase angle vary. That randomized version is more important for learning because a policy can no longer survive by memorizing one trajectory.

The Earth gravitational parameter is

$$\mu_E = 398600.4418 \text{ km}^3/\text{s}^2,$$

and the Earth radius used in the simulation is

$$R_E = 6378.137 \text{ km}.$$

The target and chaser orbital radii are therefore

$$r_T = R_E + 500 = 6878.137 \text{ km}, \qquad r_C = R_E + 485 = 6863.137 \text{ km}.$$

For a circular orbit, the speed is

$$v = \sqrt{\frac{\mu_E}{r}}.$$

This gives

$$v_T = 7.6126 \text{ km/s}, \qquad v_C = 7.6209 \text{ km/s}.$$

3 Dynamics Model

The first model uses two-body orbital dynamics with an optional thrust acceleration:

$$\ddot{\mathbf{r}} = -\mu_E \frac{\mathbf{r}}{\|\mathbf{r}\|^3} + \mathbf{a}_T.$$

Here $\mathbf{r} \in \mathbb{R}^2$ is the spacecraft position vector, and $\mathbf{a}_T$ is the commanded thrust acceleration. The target spacecraft is propagated with $\mathbf{a}_T = \mathbf{0}$. The chaser may choose one of five discrete actions:

$$\mathcal{A} = \{\text{coast}, \text{prograde}, \text{retrograde}, \text{radial-out}, \text{radial-in}\}.$$

The maximum thrust acceleration in this first run is

$$a_{\max} = 2.0 \times 10^{-5} \text{ km/s}^2 = 0.02 \text{ m/s}^2.$$

The numerical propagator is fourth-order Runge–Kutta with timestep

$$\Delta t = 10 \text{ s}.$$

4 The First Custom Formula

At each decision time t_k, the agent evaluates every candidate action $a \in \mathcal{A}$. Each candidate action is held constant over a lookahead horizon

$$H = 900 \text{ s}.$$

Let the relative position and velocity after the lookahead be

$$\boldsymbol{\rho}_H(a) = \mathbf{r}_C(t_k + H; a) - \mathbf{r}_T(t_k + H),$$

$$\boldsymbol{\nu}_H(a) = \mathbf{v}_C(t_k + H; a) - \mathbf{v}_T(t_k + H).$$

Let the closest predicted distance during the lookahead be

$$d_{\min}(a) = \min_{\tau \in [0,H]} \|\mathbf{r}_C(t_k + \tau; a) - \mathbf{r}_T(t_k + \tau)\| .$$

The first decision formula for this project is the **Rendezvous Intent Functional**:

$$\boxed{\mathcal{J}_{\mathrm{RIF}}(a; t_k) = \|\boldsymbol{\rho}_H(a)\| + \lambda_v \|\boldsymbol{\nu}_H(a)\| + \lambda_c d_{\min}(a) + \lambda_f \Delta v_a}$$

The selected action is

$$a^*(t_k) = \arg\min_{a \in \mathcal{A}} \mathcal{J}_{\mathrm{RIF}}(a; t_k).$$

4.1 Meaning of Each Term

The term $\|\boldsymbol{\rho}_H(a)\|$ rewards actions that reduce the predicted range to the target after the lookahead horizon.

The term $\lambda_v \|\boldsymbol{\nu}_H(a)\|$ penalizes high relative velocity. This matters because rendezvous is not just about getting close; the chaser must also arrive gently enough to remain controllable.

The term $\lambda_c d_{\min}(a)$ is a first-pass closeness-seeking term. In this initial no-debris scenario, smaller closest approach is useful, so this term helps the planner prefer actions that create an actual rendezvous opportunity. In future debris or docking scenarios, this term should be replaced or combined with a safety barrier that penalizes being too close at unsafe speeds.

The term $\lambda_f \Delta v_a$ penalizes fuel use. For a constant burn over the lookahead horizon,

$$\Delta v_a = \begin{cases} 0, & a = \text{coast}, \\ a_{\max} H, & a \neq \text{coast}. \end{cases}$$

The current simulation uses the unnormalized implementation

$$\mathcal{J}_{\mathrm{run}}(a; t_k) = D_H(a) + 900 V_H(a) + 0.2 D_{\min}(a) + C_f(a),$$

where D_H is in kilometers, V_H is in kilometers per second, and C_f is a small burn penalty. This is not meant to be the final objective function. It is the first testable version: simple enough to debug, but structured enough to evolve.

5 What We Are Testing

This first paper tests four basic ideas:

1. whether a short-horizon action evaluator can reduce target distance without being explicitly taught a Hohmann transfer,
2. whether relative velocity can be included in the same formula as position error,
3. whether the same physics can be implemented consistently in Python and C++,
4. whether this formula gives us a foundation that can later become a reinforcement learning reward or model predictive control objective.

6 Fixed-Scenario Result

The first simulation ran for up to 7200 s, with a new decision every 60 s. The success condition was

$$\|\boldsymbol{\rho}\| \leq 5 \text{ km} \qquad \text{and} \qquad \|\boldsymbol{\nu}\| \leq 0.020 \text{ km/s}.$$

Quantity	Value
Initial range	309.52 km
Final range	4.89 km
Minimum range	4.89 km
Final relative speed	0.0128 km/s
Approximate Δv used	80.6 m/s
Simulated time to success	90.2 min

Table 1: First autonomous rendezvous result using the Rendezvous Intent Functional.

7 Randomized Scenarios and Q-Learning

After the fixed case worked, I converted the Python simulation into a small reinforcement-learning-style environment with `reset()` and `step(action)`. The first learning agent is deliberately modest: a tabular Q-learning policy with discretized distance, radial and along-track geometry, relative velocity, closing speed, fuel remaining, and decision time. It is warm-started from the greedy planner, so it should be treated as a learning scaffold rather than an independent solution.

The important change is that the environment can now randomize the target altitude, chaser altitude offset, and initial phase angle. This immediately made the problem more honest. The randomized scenarios are staged into easy, medium, and full difficulty ranges. The greedy planner still gives a useful baseline, but the Q table by itself struggles to generalize across randomized starts.

The current policy evaluator therefore uses a one-decision projection guard. If a learned action is unknown, low-confidence, unsafe, or clearly worse than the greedy action over the next decision interval, the evaluator falls back to the greedy planner. This makes the policy safer and gives a stable benchmark, but it also means that the current Q-learning result should be read carefully: the guarded policy matches the planner, while the next goal is to make the learned table beat the planner without depending on the guard.

Difficulty	Random	Greedy	Guarded Q-learning
Easy	0/24	20/24	20/24
Medium	0/24	7/24	7/24
Full	0/24	5/24	5/24

Table 2: Current 24-seed curriculum benchmark for random, greedy, and guarded tabular Q-learning policies.

The easy benchmark improved from the earlier unguarded learned-policy result of 1/24 successes to 20/24 successes after the state representation, reward shaping, and projection guard were added. Medium improved from 2/24 successes and 52.35 km mean final range to 7/24 successes

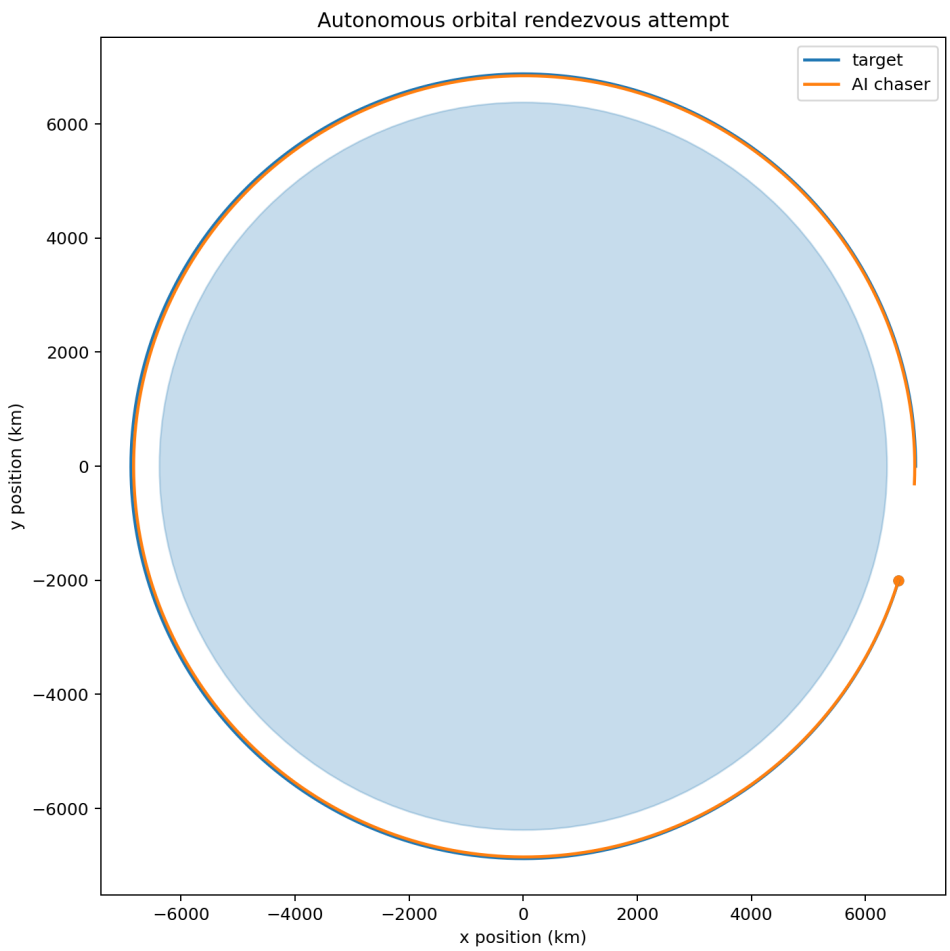

Figure 1: Target and chaser trajectories for the first autonomous rendezvous attempt.

and 19.28 km mean final range. The full benchmark currently matches greedy at 5/24 successes. This is progress, but the equality between greedy and guarded Q-learning also exposes the current limitation: the learned policy is not yet independently better than the baseline.

8 Discussion

The result is promising because a very simple action evaluator was able to reduce range from roughly 310 km to under 5 km while satisfying the relative-speed constraint. This does not mean the agent is intelligent in the final sense. It means the project now has a working decision loop:

$$\text{state} \rightarrow \text{candidate actions} \rightarrow \text{physics rollout} \rightarrow \text{score} \rightarrow \text{chosen action}.$$

That loop is exactly what we need before serious reinforcement learning. The randomized tests also show why the baseline matters. One successful Q-learning replay can look good, but the multi-seed curriculum evaluation tells the truth: the current learner is not robust enough to beat the planner yet. The projection guard is useful engineering, but it is also a diagnostic. When guarded Q-learning exactly matches greedy, the system is telling us that the learned action table

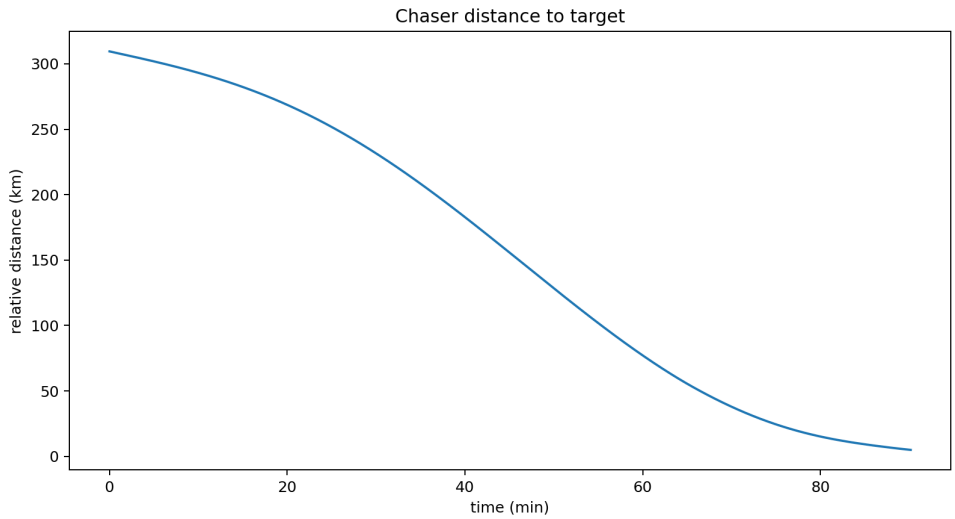

Figure 2: Relative distance between the chaser and target over time.

Difficulty	Policy	Mean range	Best range	Mean speed
Easy	Random	118.80 km	23.55 km	0.1286 km/s
Easy	Greedy	4.91 km	1.27 km	0.0124 km/s
Easy	Guarded Q-learning	4.91 km	1.27 km	0.0125 km/s
Medium	Random	120.92 km	9.75 km	0.1289 km/s
Medium	Greedy	19.28 km	2.75 km	0.0273 km/s
Medium	Guarded Q-learning	19.28 km	2.75 km	0.0273 km/s
Full	Random	152.22 km	11.50 km	0.1628 km/s
Full	Greedy	44.89 km	4.83 km	0.0560 km/s
Full	Guarded Q-learning	44.89 km	4.83 km	0.0560 km/s

Table 3: Current mean and best final ranges from the same 24-seed benchmark.

still needs better generalization. That is useful, because now the project has a way to measure progress instead of only producing good-looking animations.

9 Next Steps

The next version should introduce a normalized objective:

$$\tilde{\mathcal{J}}(a;t_k) = w_r \frac{\|\boldsymbol{\rho}_H(a)\|}{r_{\text{ref}}} + w_v \frac{\|\boldsymbol{\nu}_H(a)\|}{v_{\text{ref}}} + w_f \frac{\Delta v_a}{\Delta v_{\text{ref}}} + w_s B(a),$$

where $B(a)$ is a safety barrier term. For example, a future debris-aware version could use

$$B(a) = \sum_i \frac{1}{\max(d_i(a) - d_{\text{safe}}, \epsilon)^2},$$

where $d_i(a)$ is the predicted distance to object i, d_{safe} is the required safety distance, and ϵ prevents division by zero.

The immediate AI work is more specific:

- reduce the policy's dependence on greedy fallback and projection guards,
- improve the state bins for distance, radial position, along-track position, radial velocity, along-track velocity, closing speed, fuel, and time,
- adjust the reward so the agent learns controlled closing speed instead of only reducing distance,
- continue staged curriculum training across easy, medium, and full randomized scenarios,
- compare tabular Q-learning against a linear function approximator or another compact generalizing policy,
- compare every learner against the greedy planner, the random policy, and the fixed-scenario baseline.

After that, the project should add:

- debris avoidance,
- sensor noise,
- continuous thrust,
- low-thrust electric propulsion,
- battery and reactor power constraints,
- thermal/radiator limits,
- reinforcement learning agents trained against the same environment.

10 Conclusion

This first paper defines the foundation of the AI in Space project, but it should be read as a lab note, not a finished result. The first simulation is deliberately small, but it already combines orbital mechanics, numerical integration, autonomous action selection, measurable success criteria, and visual replay. The Rendezvous Intent Functional is the first custom formula in the project. It is not final; it is a testable starting point. The randomized Q-learning result is the more important lesson: the guarded system can now match the greedy planner across the current benchmark, but it does not yet improve on that planner. That is the next problem to solve.

11 Long-Term Aim: The Spacecraft Autonomy Functional

The long-term aim of this project is to move from a rendezvous-only score to a general mission decision formula. The formula below is not something this first simulation has proven. Instead, it is the direction we are aiming toward: a single objective that lets an autonomous spacecraft balance mission progress, fuel, safety, power, heat, uncertainty, and timing.

We call this target formula the **Spacecraft Autonomy Functional**:

$$\boxed{\begin{aligned}\mathcal{J}_{\mathrm{SAF}}(a;t) = {} & w_m \Phi_{\mathrm{mission}}(x_{t+H}) + w_r \frac{\|\boldsymbol{\rho}_H(a)\|}{r_{\mathrm{ref}}} + w_v \frac{\|\boldsymbol{\nu}_H(a)\|}{v_{\mathrm{ref}}} + w_f \frac{\Delta v_a}{\Delta v_{\mathrm{ref}}} \\ & + w_p \frac{P_{\mathrm{req}}(a)}{P_{\mathrm{avail}}} + w_q \frac{Q_{\mathrm{thermal}}(a)}{Q_{\mathrm{limit}}} + w_s \mathcal{B}_{\mathrm{safety}}(a) + w_u \mathrm{tr}(\Sigma_{t+H}(a)) + w_\tau \left| \frac{\Delta \tau_{\mathrm{rel}}(a)}{\tau_{\mathrm{ref}}} \right| .\end{aligned}}$$

The autonomous spacecraft should choose

$$a^*(t) = \arg\min_{a \in \mathcal{A}} \mathcal{J}_{\mathrm{SAF}}(a;t).$$

Each term represents a future research layer:

- Φ_{mission}: mission progress, such as inspection completion, docking readiness, transfer accuracy, or cargo delivery.
- $\|\boldsymbol{\rho}_H\|/r_{\mathrm{ref}}$: normalized future position error.
- $\|\boldsymbol{\nu}_H\|/v_{\mathrm{ref}}$: normalized future relative velocity error.
- $\Delta v_a/\Delta v_{\mathrm{ref}}$: normalized propellant or maneuver cost.
- $P_{\mathrm{req}}/P_{\mathrm{avail}}$: power demand relative to available spacecraft power.
- $Q_{\mathrm{thermal}}/Q_{\mathrm{limit}}$: thermal load relative to radiator or component limits.
- $\mathcal{B}_{\mathrm{safety}}$: collision, keep-out-zone, radiation, or hardware-limit barriers.
- $\mathrm{tr}(\Sigma_{t+H})$: predicted navigation uncertainty, where Σ is the state-estimation covariance.
- $\Delta\tau_{\mathrm{rel}}/\tau_{\mathrm{ref}}$: relativistic timing correction, useful later for precision navigation and clock-sensitive missions.

This formula gives the project a north star. The first Rendezvous Intent Functional is a small subset of it. As the simulator grows, each term can become a separate experiment: first orbital motion, then collision safety, then uncertainty, then power and thermal constraints, then nuclear-electric propulsion, and eventually relativistic navigation corrections.